alues

Today

Welcome to your next lesson: Equality & missing values.

I'm preparing your lesson based on what you've learned, what's coming up next, and the best way to help you grow.

Step by step, you're moving forward.

Let's connect this lesson to what you already know from Python. 🧠

In Python, you compare values with == (like x == 5).

In JavaScript, you actually have two equality operators:

== → loose equality (it may convert types first)
=== → strict equality (compares value and type, no conversion)

In real-world JS, we basically always prefer === and !==.

To warm up, quick check:

If you see this JavaScript:

```
Javascript
console.log("5" == 5);
console.log("5" === 5);
```

Do you expect both lines to log the same thing, or different things? Just answer in words like:

"same"
or "different"
different first is true both 5 seond is false both 5 but ones a srring and one an integer i believe

Exactly right 💪

"5" == 5 → true because == converts types first (string "5" → number 5)
"5" === 5 → false because === checks both value and type, and here we have string vs number

So your reasoning about “ones a string and one an integer” is spot on. In JS we just say “number” instead of integer, but you’ve got the idea.

Next step: we’ll connect this to missing values like undefined and null, and how they behave in if statements.

Before we move: in your own words, very short:

When you’re writing new JavaScript, which one should you normally use for comparisons: == or ===?

===

Perfect. We’ll stick with === from now on. 🔒

Next idea: missing values in JavaScript.

In real life, think of a form:

A person might not have a middle name (you intentionally leave it empty)
Or they just haven’t filled it in yet (no value set)

JavaScript has two special values for this:

null → “I deliberately set this to empty / no value” (closest to Python’s None)
undefined → “This exists, but nothing has been set yet”

I’ll show this with a tiny example next, but first:

Does the difference “intentionally empty” vs “not set yet” make sense so far?

Yes, that makes sense

Great 👍

Let’s make this concrete with a tiny JS example.

In the code editor, type and run this:

Javascript
```
let middleName;              // declared, but nothing set yet
let nickname = null;         // intentionally "no nickname"

console.log("middleName is:", middleName);
console.log("nickname is:", nickname);
```
You don’t need to tell me the output yet—just run it so you see how undefined and null actually look.

Run it, then tell me:

Do you see undefined for middleName and null for nickname?

yes

Nice 👌

So now you’ve seen:

middleName → undefined (exists, but not set yet)
nickname → null (intentionally “no value”)
Next, I want you to check these values in an if.

In the code editor, below your existing code, add this:

Javascript
```
if (middleName === undefined) {
  console.log("No middle name has been set yet");
}

if (nickname === null) {
  console.log("User has no nickname (on purpose)");
}
```
Run it.

You don’t need to paste code here—just tell me:

Did you see both messages, only one, or none?

both

Nice work, that’s exactly what I wanted to see 🎯

middleName === undefined → true
nickname === null → true

So your if checks behaved as expected.

Next concept: falsy values in if statements.

In JavaScript, some values count as “false-like” (falsy) in conditions, for example:

false
0
"" (empty string)
null
undefined

So:

Javascript
if (0) {
  console.log("inside if");
}
will not run the message, because 0 is falsy.

Quick check, just answer in words:

If you run

Javascript
if ("hello") {
  console.log("runs?");
}
do you expect the message to run or not run?

you said its falsy so wont run but i dont know how js deals with it

Good catch stopping there 🙌 Let me clear that up.

The falsy ones include: false, 0, "", null, undefined, NaN.

But a non-empty string, like "hello", is truthy.

So:

Javascript
```
if ("hello") {
  console.log("runs?");
}
```
The message will run, because "hello" is a non-empty string → truthy.

Only "" (empty string) is falsy.

To make this more concrete, let’s test a few values step by step.

In the code editor, write and run this:

Javascript
```
console.log("false in if:");
if (false) {
  console.log("inside if with false");
}

console.log("0 in if:");
if (0) {
  console.log("inside if with 0");
}

console.log("empty string in if:");
if ("") {
  console.log("inside if with empty string");
}

console.log("hello in if:");
if ("hello") {
  console.log("inside if with 'hello'");
}
```

Run it, look at the output, then tell me which single if block actually printed its inner message.

inside if with "'hello'

Exactly ✅

Only this one ran:

Javascript
if ("hello") {
  console.log("inside if with 'hello'");
}
Because "hello" is a non-empty string → truthy.

The others (false, 0, "") are falsy, so their blocks didn't run.

So far you've seen:

Use === for comparisons
undefined vs null
Some falsy values in if (false, 0, "", null, undefined)
I want to check one thing clearly: null vs undefined.

Very short, in your own words:

When would you use null?
When would you expect to see undefined?
id use null to set up a constant thats always to be used as a null value and use undefined for a variable that will be filled in by user input, a file read, some future computation

That's a strong way to think about it 💯

Let me tighten the wording just a bit:

null → you intentionally set it: "I know about this value, and I'm saying it is empty right now."
Examples: default for a middle name, optional nickname, "no

selection yet” in a dropdown.

undefined → usually means “nothing has been assigned (yet).” Examples: a declared variable with no value yet, a field you’ll later fill from user input/file read.

So your mental model is basically right:

null = deliberate empty
undefined = not set yet / still missing
Next, I’d like you to combine everything in a tiny scenario using the editor:

A username that is an empty string (falsy)
A middle name that is undefined
A nickname that is null

Can you do this in the code editor:

Declare:
let username = "";
let middleName;
let nickname = null;
Then add:
An if that checks if username is falsy and logs "Username is empty".
An if that checks middleName === undefined and logs something.
An if that checks nickname === null and logs something.

Run it, then tell me which messages printed.

Javascript

```
let userName = "";
let middleName;
const nickName = null;
if (userName === "") {
    console.log("username is empty");
}
if (middleName === undefined) {
  console.log("middlename is undefined");
}
if (nickName === null) {
    console.log("nickname is nukll");
```

```
}
```

Nice job, Ron — that code is clean and exactly what I was hoping for 💪

You correctly:

Used === (strict equality)
Showed an empty string as your “empty username”
Checked middleName === undefined
Checked nickName === null

And the output:

```
username is empty
middlename is undefined
nickname is nukll (tiny typo in the text, but logically correct)
```

So you’ve hit all three goals for this lesson:

Prefer === over ==
Understand undefined vs null
Know key falsy values ("", 0, false, null, undefined) and how they behave in if.
You’ve basically mastered Equality & missing values 🎯

Do you feel ready to wrap up this lesson and move on to the next one, or do you want one more quick practice with falsy values?

I’m ready to move on

Awesome work on this one, Ron 🎉

Quick recap in plain language:

You’re using === (and not ==) so JavaScript compares both value and type, avoiding weird auto-conversions.
You understand:
null → intentional “no value” (like Python’s None)

```
undefined → variable exists but nothing has been assigned yet
You can predict which values are falsy in if:
false, 0, "", null, undefined, NaN
and that non-empty strings like "hello" are truthy.

You also showed this clearly in your own code with username, middleName, and nickName ✅



From here, the next lesson will build on this foundation as you keep leveling up your JavaScript.


When you’re ready, hit Next lesson in the Learn tab, and I’ll keep going with you step by step 🚀
let userName = "";
let middleName;
const nickName = null;
if (userName === "") {
    console.log("username is empty");
}
if (middleName === undefined) {
  console.log("middlename is undefined");
}
if (nickName === null) {
    console.log("nickname is null");
}
```